Explanation Is the Bottleneck: Measuring and Gating XAI Cost in Real-Time Stress Detection on a Raspberry Pi Fog Node

Sena Cindioğlu
Dept. of Computer Engineering
Yıldız Technical University, Istanbul, Türkiye

***Abstract*—Post-hoc explanations are routinely attached to wearable stress classifiers, but the time needed to produce them is rarely measured and, to our knowledge, has not been measured inside a real-time physiological monitoring pipeline. We deploy a frozen 49,890-parameter 1D-CNN trained on WESAD as a TCP client–server system, run its server either co-located with the client on a laptop or on a Raspberry Pi 4 fog node, and measure the seven metrics used in fog-computing studies with an open-loop sensor client and drift-corrected clock synchronisation. Without explanation, the server pipeline takes a median of 5.7 ms per 10 s window on the laptop and 21.9 ms on the Pi. A single region-based LIME or SHAP (GradientExplainer) explanation takes a median of 203–784 ms on the laptop (four operating points) and 744–1,181 ms on the Pi (two reduced-sample operating points), i.e. 95–99% of the execution time. The median one-way network latency is at most 3.7 ms, whereas the Pi is 2.6× slower per explanation for LIME with 250 samples and 5.8× slower for SHAP with 50 samples, and every response of the latter exceeds the 1 s real-time bound. On a 43-minute segment in which 20.0% of windows had confidence below 0.80, explaining only those windows reduced the mean execution time 4.5–4.7× and raised the throughput capacity of one worker from 4 to 19 concurrent users. Gating directs explanations to errors only where confidence is calibrated: at the same threshold it explains 26 of 29 misclassified windows of one validation subject but 22 of 184 of another.**

***Index Terms*—explainable AI, fog computing, real-time systems, stress detection, LIME, SHAP, Raspberry Pi**

I. Introduction

Stress detection from wearable physiological signals is well established on public datasets such as WESAD [1], and post-hoc explanations such as LIME [2] and SHAP [3] are now a common companion to the classifier. These explanations are evaluated almost exclusively for quality. A systematic review of XAI evaluation explicitly excludes run time from its scope [4], and recent work that explains stress classifiers trained on WESAD reports neither inference nor explanation time [5]. In a real-time monitor this omission matters: a new 10 s window arrives every 5 s, and the time spent explaining one window is unavailable to the next window and to other users of the same server.

Fog-computing studies of physiological monitoring measure precisely such system-level quantities—latency, queuing delay, execution time, response time, jitter, bandwidth and resource usage—but without explanations [6], [7]. Conversely, XAI cost is usually reported without a network or a deployment. This paper measures both in one budget, on a resource-constrained fog node of the kind used in those studies. Our contributions are:

1) An end-to-end measurement of explanation cost inside a TCP client–server stress detection pipeline at two placements, using the seven metrics of [6], an open-loop load generator and drift-corrected clock synchronisation (Sec. III).
2) Evidence that explanation accounts for 95–99% of server execution time and that, once it is enabled, placement matters through compute rather than network (Secs. V-A, V-B).
3) A measured characterisation of confidence-gated explanation on the fog node—cost, throughput capacity and the queuing of several users—each matched by a simple analytical model (Secs. V-C, V-D).
4) The calibration condition under which gating directs explanations to errors, and the cost–stability trade-off between explainer operating points (Secs. V-E, V-F).

II. Related Work

Fog-based physiological monitoring. Erzurumluoğlu et al. [6] deploy emotion recognition on DREAMER with a Raspberry Pi 4 fog worker and report the seven metrics we adopt. HealthFog [7] compares placements for cardiac diagnosis in terms of latency and energy. Neither includes explanations. El Rashidy et al. [8] combine a fog architecture with LIME and Grad-CAM for emotion recognition but do not report per-window latency or explanation cost.

The cost of explanation. ShaTS [9] states an explicit real-time criterion—an explainer that needs more than 1 s per window is unsuitable for real-time use—and shows KernelSHAP violating it. Reported costs vary by orders of magnitude with hardware, input size and estimator: KernelSHAP takes seconds to minutes per sample [10], [11], whereas amortised estimators such as FastSHAP [11] trade a long training phase for millisecond explanations. On a Raspberry Pi 4, conditionally triggered SHAP has been reported to add about 195 ms per prediction in a cold-chain monitoring deployment [12].

Selective computation. Gating computation on softmax confidence is established for inference through early exits [13] and confidence cascades [14], and has been proposed for explanations as budgeted selective explanation [15] and as explanations of rejected samples [16]. We do not claim the

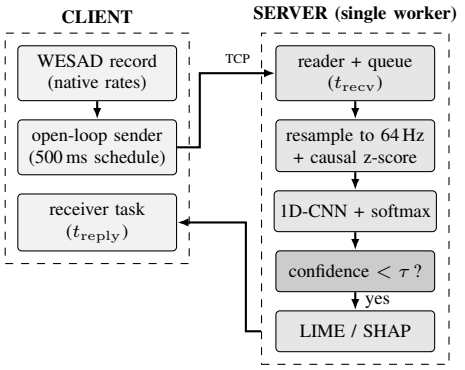


Fig. 1. Measured pipeline. The explanation is produced by the same worker as the prediction, so its cost is part of the measured execution and response times. The server runs either on the client's laptop (co-located) or on the fog node.

mechanism. We characterise it end to end in continuous physiological monitoring on constrained hardware, where its benefit depends on calibration [17].

III. System and Measurement Method

A. Model and data

The classifier is a temporal 1D-CNN with two convolution blocks ([32, 128] filters, kernel size 9, BatchNorm–ReLU–MaxPool–Dropout), global average pooling and two dense layers (49,890 parameters), trained with focal loss [18] on 14 chest and wrist channels resampled to 64 Hz, in 10 s windows with a 5 s hop. The architecture reaches 0.92 F1 under 15-fold leave-one-subject-out validation. The deployed checkpoint was trained on 13 subjects; the remaining two, S11 and S14, served as the validation set for checkpoint selection (validation F1 0.939 under offline, full-record normalisation). They were never used for training, but they are not independent test subjects; all measurements below are taken on them. The model is frozen and used unchanged throughout. A transformer from our offline study reached comparable accuracy (91.1% on a single test subject) but has 37× more parameters and needs 29× longer per window on the same CPU, partly because it consumes 700 Hz input; we therefore deploy the CNN.

B. Real-time pipeline

The client replays a WESAD record at the sensors' native rates (chest 700 Hz, wrist 4–64 Hz) in 500 ms ticks, each sent as one binary frame of 11,553 bytes. Resampling to 64 Hz is performed by the server and is part of the measured execution time. Offline training normalised every subject with statistics of the complete record, which uses future data; the server instead normalises causally (Welford's algorithm on the data seen so far). Under this constraint the deployed model reaches 0.822 accuracy and 0.694 stress F1 over 1,198 labelled windows of S11 and S14 in an offline replay, and 0.922 accuracy and 0.881 stress F1 over 372 labelled windows of the first 43 minutes of S11 on the fog node.

C. Transport and measurement

Single worker with an explicit queue. A reader task timestamps every frame on arrival (t_{recv}) and enqueues it; a single worker drains the queue (t_{start}, t_{end}). We define latency $= t_{\mathrm{recv}} - t_{\mathrm{send}}$, queuing delay $= t_{\mathrm{start}} - t_{\mathrm{recv}}$, execution time $= t_{\mathrm{end}} - t_{\mathrm{start}}$ and response time $= t_{\mathrm{reply}} - t_{\mathrm{send}}$. Jitter is the mean absolute difference of consecutive response times, as in [6]. Bandwidth is the number of bytes exchanged per second in both directions, and resource usage is the mean CPU utilisation and peak resident memory (RSS) of the server process. Nagle's algorithm is disabled (`TCP_NODELAY`, the default of Python's asyncio).

Open-loop load. A client that waits for each reply before sending the next tick is a *closed* load generator. When an explanation takes longer than one tick, the emulated sensor pauses and the waiting time is omitted from the measurement, which is the coordinated-omission problem of closed-loop benchmarking [20]. In a check on the laptop with an earlier software stack (PyTorch 2.8.0) and the 200-sample SHAP explainer, a closed-loop client delayed its sends by 661 ms at the 95th percentile of 140 packets. Our client is open-loop: a sender task emits every tick on schedule, independently of a receiver task. Its send lag on the Pi stayed below 22 ms at the 95th percentile in every single-client run.

Clocks. One-way latency requires the client and server clocks. We use monotonic clocks and estimate their offset NTP-style from the minimum-RTT exchange of 12 ping rounds at connection set-up [21]. Crystal drift made a single estimate produce negative latencies in long runs, so the offset is re-estimated every 60 s (4 rounds) and interpolated linearly for every packet. We report as uncertainty half of the median, over re-estimations, of the minimum RTT; it was at most 1.1 ms in the runs reported here.

D. Explainers and confidence gating

Region-based LIME. Each 14×640 window is divided into 20 time segments per channel (280 regions). A masked region is replaced by the channel mean; perturbed samples are weighted with an exponential kernel and fitted by ridge regression. We use 500 samples (*LIME*) and 250 samples (*LIME-cheap*). **SHAP.** SHAP's GradientExplainer (expected gradients [19]) with a fixed 100-window k-means background drawn from training subjects in the causal input space, using 200 samples (*SHAP*) and 50 samples (*SHAP-cheap*). This estimator is not KernelSHAP, whose cost structure differs [10]. All random seeds are fixed, and explainers are warmed up when the server starts, outside the measurement. The model runs with one PyTorch thread; the numerical libraries used by the explainers (notably BLAS in LIME's ridge regression) keep their default multithreading on both placements.

Gating. Let c be the maximum softmax probability of a window. The window is explained if and only if $c < \tau$; $\tau = 0$ never explains and $\tau = 1$ always explains.

TABLE I
PLACEMENTS AND SOFTWARE. THE SAME PYTORCH RELEASE RUNS ON BOTH SERVERS.

	Co-located (laptop)	**Fog**
Hardware	Intel Core 13th gen. (Raptor Lake), 20 logical cores	Raspberry Pi 4 Model B (rev. 1.1), 4× Cortex-A72, 1.5 GHz
Memory	16 GB	4 GB
OS	Windows 11	Raspberry Pi OS 64-bit (Debian 13)
Software	Python 3.11.9, PyTorch 2.14.1 (CPU), SHAP 0.51.0	Python 3.13.5, PyTorch 2.14.1 (CPU), SHAP 0.53.0
Link	loopback	LAN; Pi connected by Ethernet

IV. EXPERIMENTAL SETUP

Placements (Table I). In the *co-located* placement the client and the server run as separate processes on the same laptop and communicate over loopback; this is the lowest-latency placement available, not a wearable device. In the *fog* placement the server runs on a Raspberry Pi 4 with active cooling, the same board family as the worker of [6]. Below, "laptop" and "co-located" refer to the same placement.

Protocol. All single-client runs replay subject S11 in real time with causal normalisation; the first 15 windows of every client are discarded as warm-up. Each configuration is measured in one run. Runs with a fixed policy (no explanation, or explanation of every window) last 17 minutes (184 measured windows from the baseline segment of the record, which contains no stress); selective runs last 43 minutes (504 windows, spanning baseline and stress); multi-client runs replay S11 and S14 alternately for 5 minutes. A new decision every 5 s is a design choice for monitoring a slowly varying state; it differs from the 3 s window and 1 s hop of [6], so timing results are not directly comparable with that study. We define the *throughput capacity* of a worker as the hop divided by the mean execution time: the number of clients it can keep up with, regardless of how long an individual response takes.

V. RESULTS

A. Explanation dominates the pipeline

Table II profiles every configuration. Without explanation, the server spends a median of 5.7 ms per window on resampling, normalisation and inference on the laptop and 21.9 ms on the Pi, well under 1% of the 5 s hop. A single explanation costs a median of 203–784 ms on the laptop and 744–1,181 ms on the Pi and accounts for 95–99% of execution time in every explained configuration. Enabling an explainer raises peak server memory by a factor of 1.8–2.6 (e.g. from 245 to 522 MB on the Pi), which matters on a 4 GB node. With one client the median queuing delay is at most 0.1 ms, and bandwidth is about 187 kbps in every configuration, because explanations are computed on the server and not transmitted.

B. Where to explain: compute, not network

The network contributes little at either placement: the median one-way latency is 0.98 ms over loopback and 3.69 ms to the Pi, and at most 3.7 ms in any single-client configuration. The difference between placements comes from compute. Per explanation, the Pi is 2.6× slower than the laptop for LIME-cheap and 5.8× slower for SHAP-cheap. Because the slowdown differs between explainers, their ranking does not transfer across platforms: SHAP-cheap is the fastest explainer on the laptop but the slower of the two on the Pi. On the Pi every SHAP-cheap response exceeds 1 s (minimum 1,135 ms), violating the real-time criterion of [9], whereas LIME-cheap stays within it (95th percentile 895.3 ms), as do all four operating points on the laptop. Under explanation, the placement decision is therefore a decision about the compute capability of the node.

Three observations on the Pi qualify these numbers. First, the stages that run without explanation are slower when the processor is mostly idle: inference takes a median of 14.2 ms in the run without explanation but 7.3 ms in runs where explanations keep the CPU busy. This 1.9× difference is consistent with on-demand frequency scaling; the CPU governor was not pinned and the throttling state was not logged in these runs. Explanation runs also changed inference time on the laptop (2.5 ms without explanation, 7.1 ms with SHAP-cheap), so frequency scaling need not be the only cause. Second, LIME-cheap used on average 2.3 cores of the Pi while busy (mean CPU 35.0% against a busy fraction of 15.5%), because of multithreaded BLAS; a single-threaded LIME would be slower. Third, with LIME-cheap the median per-window difference between response and execution time is 15.1 ms on the Pi, against 5.8 ms without explanation and 3.6 ms with SHAP-cheap. This difference contains forward latency, queuing and the reply path; we did not isolate its source, and it is included in response time and jitter.

C. Selective explanation on the fog node

With $\tau = 0.80$, 101 of 504 windows (20.0%) fell below the threshold—the same windows for both explainers, because the decision depends only on the model (Table III). Mean execution time dropped 4.5× for LIME-cheap and 4.7× for SHAP-cheap, and throughput capacity rose from 6 to 29 and from 4 to 19 users. The median window is not explained (median execution 22.3 ms), so the mean is the relevant cost measure. An explained window costs as much as before: the mean explanation time in the selective runs differs from the always-explain runs by at most 0.8%, so the different segments covered by the two kinds of run do not affect the comparison. All explained SHAP-cheap windows still took more than 1 s (1,135–1,209 ms), and all explained LIME-cheap windows less (698–854 ms). Gating therefore reduces the average load, not the latency of an explained window.

Because both component costs carry over unchanged, the mean execution time of a selective run follows from them: the mean without explanation plus the explanation rate times the mean explanation time gives 173.6 and 256.1 ms against the measured 171.8 and 254.4 ms (about 1% error). Other thresholds can thus be projected from the explanation rate and the per-explanation cost. The rate itself depends on the

TABLE II

SEVEN-METRIC PROFILE PER PLACEMENT AND EXPLAINER, ONE CLIENT (MS UNLESS STATED). SERVER COLUMNS AND THE FIRST RESPONSE COLUMN ARE MEDIANS OVER WINDOWS; XAI IS THE EXPLANATION TIME WITHIN EXECUTION; CPU IS THE MEAN UTILISATION OF THE SERVER PROCESS (100% = ONE CORE); RSS IS ITS PEAK RESIDENT MEMORY.

			Server (median)				**Client**			**Resources**		
Placement	**Explainer**	**Win.**	latency	queue	exec.	xai	resp. p50	p95	jitter	CPU %	RSS MB	kbps
Co-located	none	184	0.98	0.05	5.7	–	7.2	9.3	1.4	0.5	200	187.4
	SHAP-cheap	184	0.61	0.04	213.4	203	214.5	245.3	14.5	4.7	487	187.4
	LIME-cheap	184	0.87	0.05	287.0	281	288.2	307.2	11.7	6.2	439	187.4
	LIME	184	0.83	0.05	548.1	543	549.2	589.5	20.4	11.4	451	187.3
	SHAP	184	0.91	0.05	793.1	784	794.7	842.1	25.8	16.2	519	187.3
Fog (Pi 4)	none	184	3.69	0.10	21.9	–	27.8	31.1	3.8	1.2	245	187.2
	LIME-cheap	184	3.70	0.06	755.3	744	803.2	895.3	68.8	35.0	522	187.1
	SHAP-cheap	184	1.78	0.05	1,193.3	1,181	1,197.4	1,216.4	17.2	29.4	438	187.0

TABLE III

ALWAYS VERSUS SELECTIVE ($\tau = 0.80$) EXPLANATION ON THE PI. MEANS ARE REPORTED BECAUSE SELECTIVE RUNS ARE BIMODAL. ALWAYS-RUNS COVER THE FIRST 17 MINUTES OF S11, SELECTIVE RUNS THE FIRST 43; USERS IS THE THROUGHPUT CAPACITY.

				Exec. (ms)		**Resp.**	
Explainer	**Policy**	**Win.**	**Expl.**	mean	p50	**mean**	**Users**
LIME-cheap	always	184	100%	772.8	755.3	805.2	6
	selective	504	20.0%	171.8	22.2	175.9	29
SHAP-cheap	always	184	100%	1,191.6	1,193.3	1,195.6	4
	selective	504	20.0%	254.4	22.3	258.7	19

TABLE IV

SEVERAL CLIENTS ON ONE PI WORKER, SHAP-CHEAP ON EVERY WINDOW (POOLED OVER CLIENTS, MS). UTILISATION IS $N\times$ MEAN EXECUTION / 5 S. PREDICTIONS ASSUME CLIENTS THAT START TOGETHER, SO THAT THE k-TH CLIENT SERVED WAITS BEHIND $k-1$ EXPLANATIONS.

			Resp. mean		**Resp. p95**	
Clients	**Win.**	**Util.**	meas.	pred.	meas.	pred.
1	44	24%	1,217	1,213	1,235	1,213
2	88	48%	1,790	1,783	2,400	2,377
4	176	94%	2,943	2,939	4,715	4,702

segment: over the labelled windows of the complete S11 record, the offline replay explains 14.4% at the same threshold.

D. Several users on one fog worker

Table IV runs one, two and four clients against the Pi with SHAP-cheap on every window. The clients start together, so their windows complete at the same instants and their explanations are served back to back. With N clients and mean execution time E (1,176 ms with four clients; each prediction uses the E of its own run), the response of the k-th client served is about kE. The mean response is therefore $(N+1)E/2$ and the 95th percentile approaches NE; both predictions agree with the measurements within 1.8%. At four clients the worker is 94% utilised and keeps up with the load, as the throughput capacity of 4 users predicts, but the slowest response approaches the 5 s hop. Throughput capacity is thus not a response-time guarantee: already with two clients the 95th-percentile response is 2,400 ms. Synchronised start is the worst case for this waiting; we did not measure clients with random phases.[1]

TABLE V

EXPLANATION RATE AND MISCLASSIFIED WINDOWS THAT ARE EXPLAINED, PER THRESHOLD (LABELLED WINDOWS OF THE COMPLETE RECORDS, CAUSAL INPUT SPACE, UNCALIBRATED SOFTMAX).

	S11 (ECE 0.047)		**S14** (ECE 0.227)	
τ	explained	errors caught	explained	errors caught
0.70	10.4%	18/29	4.4%	11/184
0.80	14.4%	26/29	11.8%	22/184
0.90	21.4%	29/29	29.3%	53/184

E. The calibration condition

Gating reduces cost on any subject, but it directs explanations to errors only if confidence is calibrated [17]. The model is equally confident on both validation subjects (mean confidence 0.909 and 0.908) but correct on 0.952 and 0.690 of their labelled windows, respectively (Table V). At $\tau = 0.80$ the explanation rates are similar (14.4% and 11.8%), yet 26 of S11's 29 errors are explained but only 22 of S14's 184, because S14 is often wrong with high confidence. Temperature scaling fitted on the other subject made calibration worse for both (ECE 0.256 and 0.276), since the two subjects are miscalibrated in opposite directions. The cost reduction of gating does not depend on calibration, whereas its error coverage does; with two subjects we report this as an observation, which motivates per-user calibration.

F. Explainer quality versus cost

Quality was measured offline on the laptop in the causal input space with an earlier software stack (PyTorch 2.8.0); unlike time, it does not depend on the hardware. No operating point dominates in faithfulness: for every pair the difference

[1]These runs used a server version that computed on the event loop, so waiting packets were timestamped late and waiting appeared as latency rather than queuing delay; the client also awaited its periodic clock re-synchronisation in the sender task, delaying single sends by up to 3.7 s. Response time, the only metric reported here, is measured on the client and is unaffected by the first issue. Both issues are fixed in the released code.

TABLE VI
EXPLAINER OPERATING POINTS: MEDIAN EXPLANATION TIME ON THE LAPTOP AND QUALITY ON CLASS-BALANCED WINDOWS OF S11 (FAITHFULNESS = INSERTION MINUS DELETION AUC ± STANDARD ERROR, 40 WINDOWS; MONOTONICITY = FRACTION OF NON-DECREASING STEPS OF THE INSERTION CURVE; SENSITIVITY = ATTRIBUTION CHANGE UNDER INPUT NOISE, 10 WINDOWS × 3 DRAWS, LOWER IS BETTER).

Explainer	xai (ms)	Faithfulness	Monoton.	Sensitivity
SHAP-cheap	203	0.250 ±0.043	0.565	0.123
LIME-cheap	281	0.209 ±0.035	0.604	0.049
LIME	543	0.232 ±0.037	0.579	0.045
SHAP	784	0.263 ±0.044	0.549	0.128

is smaller than the sum of the standard errors (Table VI). The explainers differ in stability instead: under small input noise, LIME's attributions change 2.7× less than those of SHAP. The cheap operating points lose no measurable faithfulness. Since LIME-cheap is also the faster explainer on the Pi (Sec. V-B), it is both the cheaper and the more stable choice on the fog node.

VI. Discussion and Limitations

Where the architectural decision moves. Without explanation, both placements answer well within the hop, and the choice between them is a latency choice, as in [6]. With explanation, two other parameters dominate: the compute capability of the node, which changes the cost of an explanation by up to 5.8×, and the triggering policy, which changes the mean cost by 4.5–4.7× on our segment. On a Raspberry Pi 4, explaining every window meets the 1 s criterion for a single user with LIME-cheap but not with SHAP-cheap; the throughput capacity of one worker is 4–6 users when every window is explained and 19–29 with gating, while individual explained responses keep their single-user latency only when they do not collide.

Comparison with the reference system. Only bandwidth is directly comparable, because the hardware and windowing differ: streaming the raw multichannel signal costs 187 kbps, whereas sending 24 extracted features at 32 requests per second costs 373.9 kbps in [6].

Limitations. (i) All measurements use the two validation subjects of the deployed checkpoint, and calibration is studied on these two only. (ii) Each configuration is a single run; within-run statistics rest on 44–504 windows, and between-run variance is not reported. In an earlier offline benchmark with 4 alternating runs on the laptop, without network or explanation, run medians varied by 2.9% and 95th percentiles by 12.3%; the effects we draw conclusions from (2.6–5.8× between placements, 4.5–4.7× from gating) are far larger. (iii) The placements share the PyTorch release but differ in processor architecture, operating system, Python and SHAP version; the placement ratio reflects the whole platform, including multithreaded BLAS and the unpinned CPU governor of the Pi (Sec. V-B). (iv) The gains of gating were measured on one segment with a 20.0% explanation rate; they scale with the rate. (v) One worker per server; a cloud placement was measured earlier with a closed-loop client and a different software stack and is not compared here. (vi) The system replays recorded data at native rates; no user study of explanation usefulness was performed.

VII. Conclusion

Measured end to end, explanation and not inference is the bottleneck of real-time explainable stress detection: it takes 95–99% of server execution time, and on a Raspberry Pi 4 fog node every 50-sample SHAP explanation takes more than one second. Confidence-gated explanation reduced the mean cost 4.5–4.7× on a segment where 20.0% of windows were explained and raised the throughput capacity of a fog worker by a similar factor, but it directs explanations to errors only where the model is calibrated. The measurement harness, including the open-loop client and drift-corrected clock synchronisation, will be made publicly available.

References

[1] P. Schmidt, A. Reiss, R. Duerichen, C. Marberger, and K. Van Laerhoven, "Introducing WESAD, a multimodal dataset for wearable stress and affect detection," in *Proc. ACM ICMI*, 2018, pp. 400–408.

[2] M. T. Ribeiro, S. Singh, and C. Guestrin, "'Why should I trust you?': Explaining the predictions of any classifier," in *Proc. ACM SIGKDD*, 2016, pp. 1135–1144.

[3] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems (NIPS)*, 2017, pp. 4765–4774.

[4] M. Nauta *et al.*, "From anecdotal evidence to quantitative evaluation methods: A systematic review on evaluating explainable AI," *ACM Comput. Surv.*, vol. 55, no. 13s, Art. 295, 2023.

[5] A. Almadhor *et al.*, "Explainable deep learning for multimodal physiological stress and emotion recognition," *Front. Bioeng. Biotechnol.*, 2025, doi:10.3389/fbioe.2025.1659002.

[6] Ö. F. Erzurumluoğlu, K. Özlem, A. T. Atalay, Ö. Atalay, and G. İnce, "Fog computing-based real-time emotion recognition using physiological signals," in *Proc. 27th Int. Conf. Advanced Communications Technology (ICACT)*, 2025, pp. 442–450.

[7] S. Tuli *et al.*, "HealthFog: An ensemble deep learning based smart healthcare system for automatic diagnosis of heart diseases in integrated IoT and fog computing environments," *Future Gener. Comput. Syst.*, vol. 104, pp. 187–200, 2020.

[8] N. El Rashidy *et al.*, "An explainable fog-based emotion recognition framework," *Front. Comput. Sci.*, vol. 7, 2025.

[9] M. Franco de la Peña *et al.*, "ShaTS: A Shapley-based method for time-series explainability," arXiv:2506.01450, 2025.

[10] A. Warnecke, D. Arp, C. Wressnegger, and K. Rieck, "Evaluating explanation methods for deep learning in security," in *Proc. IEEE EuroS&P*, 2020, pp. 158–174.

[11] N. Jethani, M. Sudarshan, I. Covert, S.-I. Lee, and R. Ranganath, "FastSHAP: Real-time Shapley value estimation," in *Proc. ICLR*, 2022.

[12] T. Taguta *et al.*, "Conditional SHAP for on-device cold-chain monitoring," arXiv:2609.14036, 2026.

[13] S. Teerapittayanon, B. McDanel, and H. T. Kung, "BranchyNet: Fast inference via early exiting from deep neural networks," in *Proc. ICPR*, 2016, pp. 2464–2469.

[14] K. Berestizshevsky and G. Even, "Dynamically sacrificing accuracy for reduced computation: Cascaded inference based on softmax confidence," in *Proc. ICANN*, 2019, pp. 306–320.

[15] L. M. Paes, D. Wei, and F. P. Calmon, "Selective explanations," arXiv:2405.19562, 2024.

[16] A. Artelt, R. Visser, and B. Hammer, "'I do not know! but why?' — Local model-agnostic example-based explanations of reject," *Neurocomputing*, vol. 558, 2023.

[17] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proc. ICML*, 2017, pp. 1321–1330.

[18] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," in *Proc. IEEE ICCV*, 2017, pp. 2980–2988.

[19] G. Erion, J. D. Janizek, P. Sturmfels, S. M. Lundberg, and S.-I. Lee, "Improving performance of deep learning models with axiomatic attribution priors and expected gradients," *Nature Machine Intelligence*, vol. 3, pp. 620–631, 2021.
[20] B. Schroeder, A. Wierman, and M. Harchol-Balter, "Open versus closed: A cautionary tale," in *Proc. USENIX NSDI*, 2006.
[21] D. L. Mills, "Internet time synchronization: The network time protocol," *IEEE Trans. Commun.*, vol. 39, no. 10, pp. 1482–1493, 1991.